Automated Software Engineering (2026) 33:5
https://doi.org/10.1007/s10515-025-00546-0

HMF: Enhancing reentrancy vulnerability detection and repair with a hybrid model framework

Mengliang Li[1] · Qiang Shen[2] · Xiaoxue Ren[1] · Han Fu[3] · Zhuo Li[4] · Jianling Sun[3]

Received: 7 February 2025 / Accepted: 27 July 2025 / Published online: 13 September 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract
Smart contracts have revolutionized the credit landscape. However, their security remains intensely scrutinized due to numerous hacking incidents and inherent logical challenges. One well-known issue is reentrancy vulnerability, exemplified by DAO attacks that lead to substantial economic losses. Previous approaches have employed rule-based and deep learning-based (DL) algorithms to detect and repair reentrancy vulnerability. Large language models (LLM) have been distinguished in recent years for their excellent understanding of text and code. However, less attention has been paid to LLM-based reentrancy vulnerability detection and repair, and direct prompt-based approaches often suffer from inefficiencies and high false positives. To overcome the above shortcomings, this paper proposes a hybrid model framework combining LLM with DL to enhance the detection and repair of reentrancy vulnerabilities. This unified framework comprises three crucial phases: the data processing phase, the vulnerability detection phase, and the vulnerability repair phase. Extensive experimental results validate the superiority of our approach over state-of-the-art baselines, and ablation studies demonstrate the effectiveness of each component. Our approach demonstrates significant improvements in vulnerability detection, with increases of 3.51% in accuracy, 2.31% in recall, 0.42% in precision, and 0.85% in F1-score. Furthermore, our approach can achieve a notable 9.62% enhancement in the repair rate. Finally, we also conducted a user study to emphasize its potential to fortify the security of smart contracts.

Keywords Smart Contract · Vulnerability Detection · Vulnerability Repair · LLM · Reentrancy

Extended author information available on the last page of the article

1 Introduction

Smart contracts are self-executing programs that run on blockchain platforms, automating the management of data and assets within these systems (Pan et al. 2023). While the adoption of smart contracts has surged across diverse sectors, security remains a critical concern, exacerbated by frequent hacking incidents and the inherent complexity of their logic (Chen et al. 2020). For instance, a smart contract vulnerability, such as the one exploited in the DAO attack (Zheng et al. 2023), has resulted in financial losses amounting to $150 million. The SWC Registry catalogs 37 distinct types of vulnerabilities, and the NCC Group identifies the top 10 most critical risks (Zheng et al. 2023). Reentrancy is particularly notorious among these, having been the principal factor behind the DAO attack (Zhang, et al. 2023). Consequently, identifying and repairing reentrancy vulnerabilities is paramount for improving the overall security of smart contracts.

Previous approaches employ rule-based (Feist et al. 2019; Kalra et al. 2018; Zhang et al. 2019) and deep learning-based (DL- based) (Shakya et al. 2022) algorithms to detect and repair reentrancy vulnerability. The representative tools include Oyente (Luu et al. 2016), Slither (Feist et al. 2019), Securify (Tsankov et al. 2018), SmartCheck (Tikhomirov et al. 2018), SliSE (Wang et al. 2024), Mythril (Ferreira et al. 2021), Rechecker (Qian et al. 2020), EVMPatch (Rodler et al. 2021), TMP (Zhuang et al. 2020), AME (Liu et al. 2021a), and CGE (Liu et al. 2021b). For instance, Zhuang et al. (2020) proposed a graph neural network-based (GNN-based) method for detecting various smart contract vulnerabilities, which transforms basic functions, relevant variables, and callback functions into key node features. Liu et al. (2021a) introduced a smart contract vulnerability detection method that integrates expert knowledge with graph neural networks, enabling the extraction of both global graph vulnerability features and local expert features.

Recent years have witnessed the remarkable success of LLM in the fields of artificial intelligence and software engineering (Chen et al. 2024; Hu et al. 2023; Fu et al. 2023; David et al. 2023; Ren et al. 2023). However, comparatively less attention has been directed toward utilizing LLMs for smart contract security. Despite the growing interest in applying LLM-based techniques to detect and repair vulnerabilities, such as reentrancy, their performance in this domain remains unsatisfactory. For example, Chen et al. (2024) conducted an empirical study examining the effectiveness of ChatGPT in identifying vulnerabilities within smart contracts. The study revealed that while ChatGPT demonstrates a high recall rate, its precision in accurately identifying vulnerabilities is relatively limited, indicating significant room for improvement in applying LLMs to smart contract security.

Several challenges remain while employing an LLM-based approach to detect and repair reentrancy vulnerability. These include: (1) **Inadequate Model Fine-Tuning in Detection Phase:** LLM is trained with the super-scale general dataset, and the smart contract-based model fine-tuning is inadequate. Large and unrelated datasets can lead to unsatisfactory performance (see Fig. 1) in vulnerability detection tasks (Chen et al. 2024). This motivates us to collect and train a domain-specific smart contract-based DL model rather than the prompt-based LLM-driven approach to detect reentrancy vulnerability. (2) **Limited Smart Contract Knowledge in Repair**

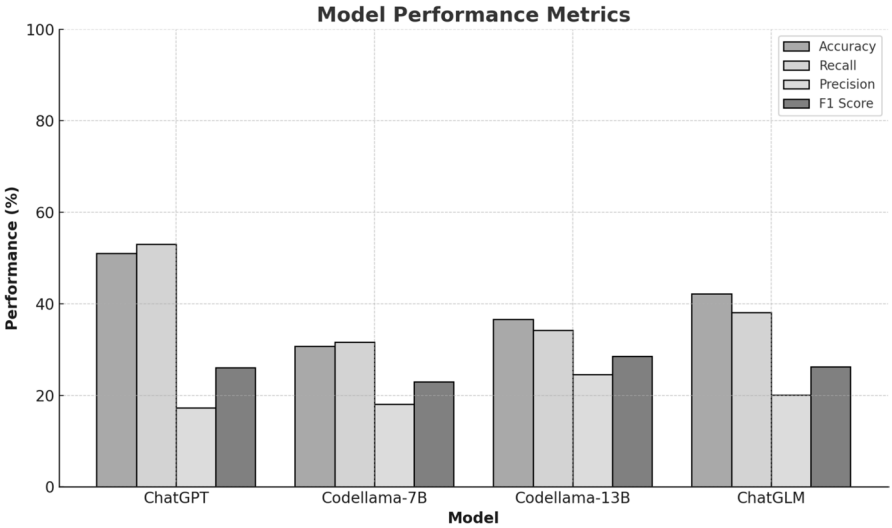

Fig. 1 Poor reentrancy vulnerability detection in four LLM-based models

```
1.function withdraw(uint amount) {
2.   if (credit[msg.sender]>= amount) {
3.     bool res = msg.sender.call.value(amount)();
4.     credit[msg.sender]-=amount;
5.   }
6.}
```
(a) Vulnerable

```
1.function withdraw(uint amount) {
2.   if (credit[msg.sender]>= amount) {
3.     credit[msg.sender]-=amount;
4.     bool res = msg.sender.call.value(amount)();
5.   }
6.}
```
(b) Non-Vulnerable

Fig. 2 (a) The vulnerable smart contract with reentrancy vulnerability. (b) The repaired smart contract with CEI

Phase: The simple prompt-based ChatGPT (Chen et al. 2024; Hu et al. 2023; Fu et al. 2023; David et al. 2023) lacks enough smart contract vulnerability knowledge, such as code structural information and reentrancy background knowledge. For instance, check-effects-interactions (CEI) is an effective but complex approach to repair reentrancy vulnerability by first updating the credit and then transferring the amount (see Fig. 2). This motivates us to construct a rich knowledge-aware prompt to repair reentrancy vulnerability.

In this paper, we present a **h**ybrid **m**odel **f**ramework (HMF) that combines DL-based detection with LLM-based repair to address reentrancy vulnerabilities. First, we introduce an enhanced DL-based model featuring advanced feature extraction, data balancing techniques, and a robust DL network to tackle the first challenge. Next, we propose a knowledge-aware, prompt-based approach to address the second challenge effectively. Specifically, our framework encompasses three crucial phases (see Fig. 3): the data processing phase, the vulnerability detection phase, and the vulnerability repair phase. (1) **Data Processing Phase**: This phase introduces a novel feature extraction algorithm to capture control-flow features of reentrancy vulnerability (see data processing phase in Fig. 3). Additionally, we propose a novel algorithm to augment imbalanced datasets while maintaining the semantic and syntactic integ-

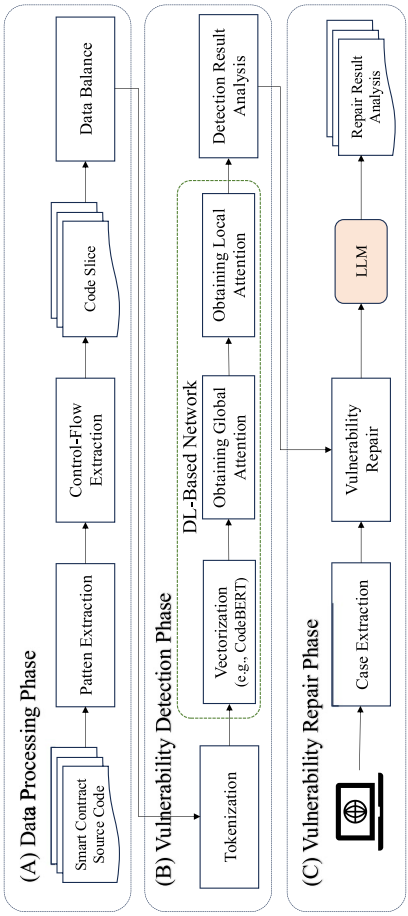

Fig. 3 The overall framework of our approach has three crucial phases: data processing, vulnerability detection, and vulnerability repair

rity of code slices. (2) **Vulnerability Detection Phase**: In this phase, we construct the DL-based network with global attention and local attention to train vulnerability features (see vulnerability detection phase in Fig. 3). This approach can overcome the high false positives while directly employing the prompt-based approach to detect reentrancy vulnerability. (3) **Vulnerability Repair Phase**: In this phase, we propose a knowledge-aware prompt-based approach to enhance LLMs' ability to repair reentrancy vulnerabilities. This prompt-based approach is constructed with vulnerable examples, repaired examples, repair approaches, and DL-based detection result analysis (see vulnerability repair phase in Fig. 3). This approach enables LLMs to autonomously learn how to fix the input code slice with pre-domain reentrancy knowledge. The DL-based detection result analysis can aid LLM in locating and repairing reentrancy vulnerability accurately.

Extensive experiments have demonstrated that our method enhances reentrancy vulnerability detection and repair capabilities, showcasing its immense potential in safeguarding smart contract security. (1) **Performance Improvement**: Our method surpasses other advanced benchmark methods, including rule-based, DL-based, and basic LLM-based approaches (see Sect. 4.2). It improves the accuracy, recall, precision, and F1-score of vulnerability detection by 3.51%, 2.31%, 0.42%, and 0.85%, respectively (see Table 1). Furthermore, our method achieves a 9.62% improvement in the repair rate (see Table 2). (2) **Ablation Study**: Comparative experiments on

Table 1 The overall comparison of smart contract reentrancy vulnerability detection

Method		Accuracy(%)	Recall(%)	Precision(%)	F1 Score(%)
LLMs-Based	GPT-3.5-Turbo	50.96	52.94	17.31	26.09
	Codellama-7B	30.74	31.61	18.06	22.99
	Codellama-13B	36.60	34.19	24.53	28.56
	ChatGLM	42.24	38.15	20.06	26.29
	DeepSeek-R1	42.24	96.77	44.12	60.61
	Starcoder2-15B	31.47	100.00	24.03	38.75
	Qwen2-72B	35.78	100.00	31.37	47.76
	LLaMA3.1-70B	29.36	100.00	29.36	45.39
Rule-Based	Security	72.89	73.06	68.4	70.41
	Smartcheck	52.00	24.32	31.03	27.07
	Osiris	56.73	63.88	40.94	49.90
	Mythril	60.00	39.21	68.96	50.00
	Oyente	71.50	50.84	51.72	51.28
	Slither	74.02	73.50	74.44	73.97
	SliSE	-	96.77	68.18	80.00
DL-Based	MLP	78.39	75.66	80.06	77.77
	RNN	79.81	74.85	82.92	78.56
	GCN	77.85	78.79	70.02	74.15
	TMP	84.48	82.63	74.46	78.11
	AME	90.19	89.69	74.06	87.94
	CGE	89.15	87.62	85.24	86.41
	ReVulDL	-	93.00	92.00	93.00
Our approach	HM-RVDR	93.7(3.51↑)	95.31(2.31↑)	92.42(0.42↑)	93.85(0.85↑)

Table 2 The overall comparison of smart contract reentrancy vulnerability repair: FR represents the fixed rate, and CR represents the correct rate

Method		Fixed	FR(%)	Correct	CR(%)
Rule-Based	SmartFix	46/52	88.46	46/52	88.46
	SGUARD	33/52	63.46	29/52	55.77
	Elysium	35/52	67.31	-	-
LLM-based	CodeLlama-7B-0shot	10/52	19.23	4/52	7.69
	CodeLlama-7B-2shot	15/52	28.85	6/52	11.54
	CodeLlama-13B-0shot	18/52	34.62	6/52	11.54
	CodeLlama-13B-2shot	25/52	48.07	10/52	19.23
	CodeLlama-13B-instruct-0shot	33/52	63.46	20/52	38.46
	CodeLlama-13B-instruct-2shot	38/52	73.07	24/52	46.15
	ChatGLM-0shot	30/52	57.69	8/52	15.38
	ChatGLM-2shot	35/52	67.31	14/52	26.92
	GPT-3.5-Turbo-0shot	36/52	69.23	30/52	57.69
	GPT-3.5-Turbo-2shot	44/52	84.61	42/52	80.77
Our Approach		49/52	94.23(9.62↑)	47/52	90.38(9.61↑)

each component demonstrate (see Sect. 4.3) that our method effectively addresses issues such as feature extraction, data balance, DL-based networks, and knowledge-aware prompts. (3) **User Study**: The user study also demonstrates the effectiveness of our approach and can significantly enhance reentrancy detection and repair (see Sect. 4.4).

We summarize our contributions as follows:

1. **Hybrid Model Framework**: We propose a hybrid model-driven framework to detect and repair reentrancy. This is the first work to integrate enhanced DL-based vulnerability detection with LLM-based vulnerability repair for the reentrancy study.
2. **Experiment Performance**: Extensive experimental results have demonstrated that our approach outperforms state-of-the-art baselines in both vulnerability detection and repair. Specifically, our method enhances the accuracy, recall, precision, and F1-score of vulnerability detection by 3.51%, 2.31%, 0.42%, and 0.85%, respectively. Additionally, it achieves a 9.62% improvement in the repair rate.
3. **User Study**: We conduct a user study to assess the effectiveness of our approach. The results indicate that our method significantly improves detection by 33.3%, repair by 56.7%, and perceived usefulness by 50%. Furthermore, our approach reduces time consumption by 14.9%.
4. **Open Source**: We open-source our replication package, including our approach's dataset and source code, for follow-up works.

2 Challenge and motivation

Previous studies have witnessed some achievements in reentrancy vulnerability detection and repair (Zhuang et al. 2020; Liu et al. 2021b; Ren et al. 2023). However, several challenges remain to construct an LLM-based framework for reentrancy vulnerability detection and repair.

- **Challenge 1: Inadequate Model Fine-tuning in Vulnerability Detection** LLM is trained with the super-scale general dataset, and smart contract-based model fine-tuning is inadequate. Large and unrelated information can disturb LLM's understanding of vulnerability features and lead to unsatisfactory performance in reentrancy vulnerability detection (Chen et al. 2024) (see Fig. 1). Previous studies have demonstrated that training domain-specific DL-based models can effectively detect smart contract vulnerability (Zhuang et al. 2020; Liu et al. 2021a; Ren et al. 2023). Therefore, this motivates us to collect and train a domain-specific smart contract-based DL model rather than utilizing the prompt-based approach to detect reentrancy.
- **Challenge 2: Inadequate knowledge Injection in Vulnerability Repair** The contemporary approach integrates ChatGPT with simplistic prompts (Chen et al. 2024; Hu et al. 2023; Fu et al. 2023) with- out adequate external smart contract vulnerability knowledge and code structural analysis. This approach can result in an LLM's inadequate understanding of reentrancy vulnerability. For instance, CEI is an effective but complex approach to repair reentrancy vulnerability by first updating the credit and then transferring the amount. Figure 2 (a) shows a vulnerable code slice due to it first wrongly transferring amount and then updating the credit. Figure 2 (b) shows a repaired code slice with a CEI approach. Motivated by in-context learning (Dong, et al. 2022) that few examples can enhance LLM's ability on domain-specific tasks, we construct a rich knowledge-aware prompt with vulnerability knowledge and code structural analysis to effectively repair reentrancy.

Our Motivation: The challenges above motivate us to create a unified framework to enhance the detection and repair of reentrancy vulnerability. To address these challenges, we propose the following approaches:

(i) **Approach for challenge 1:** We propose a powerful domain-specific DL-based model for reentrancy vulnerability detection rather than employing a prompt-based approach to detect reentrancy vulnerability. This model utilizes the feature extraction algorithm, the data balance algorithm, and the DL-based network to enhance reentrancy vulnerability detection performance.
(ii) **Approach for challenge 2:** We suggest a knowledge-based prompt-based approach for reentrancy vulnerability repair. We analyze the detection result

from the DL- based detection model and extract vulnerable code lines. Then, we construct a knowledge-aware prompt with external reentrancy vulnerability and code structural analysis to repair the vulnerability.

3 Approach

Figure 3 illustrates the overall framework of our hybrid model. This unified framework comprises three crucial phases: data processing, vulnerability detection, and vulnerability repair. Figure 4 shows an illustrated example process of our approach that transfers source code to the repair report.

- **Data Processing Phase**: We first perform a pattern extraction and then propose a novel control-flow extraction algorithm to obtain code slices. Finally, we introduce a data balance algorithm to augment the imbalanced datasets.
- **Vulnerability Detection Phase:** We construct a deep learning network with global attention and local attention to detect vulnerability. Then, we analyze the detection result and extract the vulnerable code lines for the repair phase.
- **Vulnerability Repair Phase:** We extract and summarize the smart contract's vulnerable and non-vulnerable cases from the documentation and website. Then we construct a vulnerability repair prompt with cases, approaches, and vulnerable code lines for the LLM and obtain the final repair report.

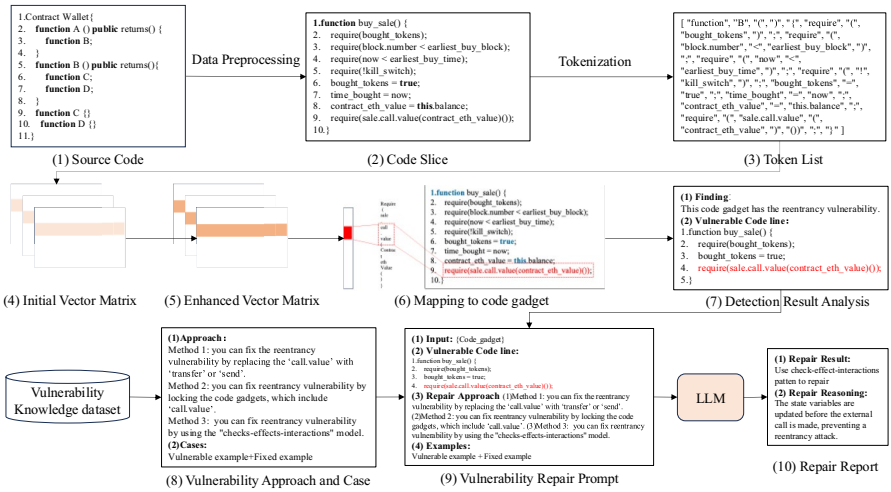

Fig. 4 The illustrated example process of our approach: from source code to repair report

3.1 Data processing phase

3.1.1 Pattern extraction

The root causes of reentrancy vulnerabilities are associated with DAO attacks (Xue et al. 2020), a type of external hacker attack targeting the logic of smart contracts. Three conditions must be met for a reentrancy vulnerability to occur: (1) The attacked smart contract must include the $call.value$ keyword, which is essential for reentrancy vulnerabilities. (2) To exploit the vulnerability, the external attacker must directly or indirectly invoke the function containing the $call.value$ keyword (Zhang et al. 2019). (3) The external attacker must con- struct a fallback function to call the function containing the $call.value$ keyword. Thus, identifying the code slices with the $call.value$ keyword and function-call relationship enable the extraction of reentrancy vulnerability features.

3.1.2 Control-flow extraction

In this section, we propose a new algorithm to extract code slices containing control-flow features of reentrancy vulnerability. We first introduce two definitions and then thoroughly analyze the vulnerability extraction algorithm.

Definition 1: (File, Function, Line, Token).

Smart contract datasets contain multiple files $F = \{f_1, f_2, f_3, \ldots, f_i\}$. Each smart contract source code contains multiple functions $Fun = \{fun_1, fun_2, fun_3, \ldots, fun_j\}$. Each function contains multiple lines of code, and each line contains multiple tokens.

Definition 2: (Core Function, Forward Function, Backward Function).

A core function contains the keyword $call.value$ associated with reentrancy vulnerabilities. The forward function calls the core function, while the core function calls the backward function. Extracting these functions is to obtain features related to reentrancy vulnerabilities and the calling relationships between functions.

Algorithm:

Algorithm 1 details how to extract multi-function code slices containing vulnerability features from smart contract source code. The input is a source code dataset containing multiple smart contracts, and the output is a set of smart contract code slices containing vulnerability features. (1) First, we define and initialize the variables for the entire function. In line 1, we define the total code slice output set Out as empty. In line 3, we define the multi-feature function set oi for each smart contract as empty, the function set funyes containing reentrancy vulnerability keywords as empty, the function set funno not containing reentrancy vulnerability keywords as empty, the forward function set funforward as empty, and the backward function set funbackward as empty. (2) In line 6, we use a function slicing algorithm to obtain the set of all functions Fun for each smart contract. (3) In lines 7–15, we extract the function set funyes containing vulnerabilities and the function set funno without containing vulnerabilities from the function set Fun based on the presence of keywords. (4) In lines 16–24, we extract the forward function set funforward and the backward function set funbackward from the function set funno based on function call relationships. (5) In line 25, we add the functions funforward and funbackward correspond-

ing to a smart contract to the function set Oi. In line 26, we add each function set Oi to the total code slice output set Oout. Finally, the entire function outputs the total code slice output set OOut.

Algorithm 1 Extracting the control-flow-based code slices of reentrancy vulnerability from smart contract source code.

Input: Smart contract set $F = (f_1, f_2, f_3, ..., f_i)$
Parameter: function set Fun, function set F_{yes} with keyword (call.value), function set F_{no} without keyword, keyword set K
Output: Code slice set $Out = (O_1, O_2, O_3 ... O_i)$

1: $Out \leftarrow \emptyset$
2: **for** each file $f_i \in FUN$ **do**
3: $O_i \leftarrow \emptyset$
4: $F_{yes} \leftarrow \emptyset$, $F_{no} \leftarrow \emptyset$
5: $F_{forward} \leftarrow \emptyset$, $F_{backward} \leftarrow \emptyset$
6: Leveraging segmentation algorithm to generate function set Fun from f_i
7: **for** each function $fun_j \in Fun$ **do**
8: **for** each keyword $k \in K$ **do**
9: **if** fun_j match k **then**
10: $F_{yes} \leftarrow F_{yes} \cup fun_j$
11: **else**
12: $F_{no} \leftarrow F_{no} \cup fun_j$
13: **end if**
14: **end for**
15: **end for**
16: **for** each function $fun_j \in F_y$ **do**
17: **for** each function $fun_k \in F_n$ **do**
18: **if** function fun_k invoke fun_j **then**
19: $F_{forward} \leftarrow F_{forward} \cup fun_k$
20: **else**
21: $F_{backward} \leftarrow F_{backward} \cup fun_k$
22: **end if**
23: **end for**
24: **end for**
25: Adding the function sets F_{yes}, $F_{forward}$ and $F_{backward}$ into O_i
26: $O_{out} \leftarrow O_{out} \cup O_i$
27: **end for**
28: **return** Out;{the set of control-flow-based code slices}

3.1.3 Data balance

We should balance the extracted code slice dataset before feeding it into the deep learning network due to the imbalanced dataset associated with the overfitting problem. In this section, we propose a novel data balance algorithm to enhance the dataset from the token-based and sentence-based levels.

Figure 5 details two approaches for augmenting the imbalanced datasets. Sentence- level insertion involves randomly selecting a line of code from a smart con-

```
1. function withdraw(uint256 amount) public {
2.     require(balances[msg.sender] >= amount, "Insufficient balance");
3.     bool success = false;
4.     success=msg.sender.call.value(amount);
5.     require(success, "Withdrawal failed");
6.     balances[msg.sender] -= amount;
7.}
```

(A) Source Code

```
1. function withdraw(uint256 amount) public {
2.     require(balances[msg.sender] >= amount, "Insufficient balance");
3.     bool success = false;
4.     success=msg.sender.call.value(amount);
5.     require(success, "Withdrawal failed");
6.     balances[msg.sender] -= amount;
7.}
8.balances
```

(B) Token-Level Insertion

```
1.function withdraw(uint256 amount) public {
2.     require(balances[msg.sender] >= amount, "Insufficient balance");
3.     bool success = false;
4.     success=msg.sender.call.value(amount);
5.     require(success, "Withdrawal failed");
6.     balances[msg.sender] -= amount;
7.}
8. bool success = false;
```

(C) Sentence-Level Insertion

Fig. 5 The two approaches augment the imbalanced datasets from sentence-level and token-level

tract code slice and inserting it at the end of the code slice. For instance, in Fig. 5, we randomly select the fourth line of code from the function withdraw and place it at the end of the entire code slice. Token-level insertion involves randomly selecting a token and inserting it at the beginning or end of the code slice without changing the overall logical structure. For instance, in Fig. 5, we randomly select the token from the function withdraw and insert it at the end of the code slice. Overall, these two approaches allow us to enhance an imbalanced dataset while preserving the semantic and syntactic structure of the code slices.

3.2 Vulnerability detection phase

3.2.1 DL-based network

We propose a deep learning-based (DL-based) neural network to train the enhanced code slice dataset. We draw inspiration from pre-trained models, transformer encoders, and convolutional neural networks (CNN) to design the neural network in Fig. 6.

These deep learning neural networks include four parts: vulnerability code representation based on a cross-language pre-trained layer, global vulnerability feature extraction based on encoder layers, local feature extraction based on CNN layers, and

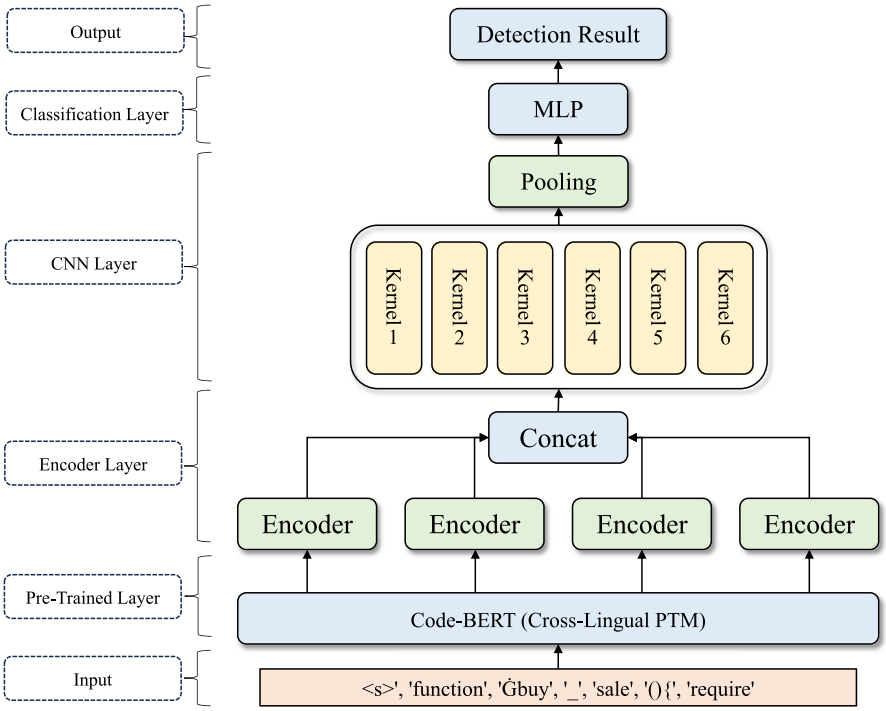

Fig. 6 The overall structure of the deep learning (DL-based) network

a classification layer based on an MLP. (1) Pre-Trained Layer: We utilize cross-language pre-trained code representation models (Code-PTM, Code Pre-Trained Models) to vectorize the Solidity language of smart contracts. Since no pre-trained models specifically for Solidity exist, we choose pre-trained models trained in other languages to represent smart contract code slices. Pre-trained code representation models in our approach include CodeBERT, RoBERTa, and CodeBERTa. (2) Encoder Layer: We utilize transformer encoders to extract global vulnerability features of reentrancy vulnerabilities. The core principle of the encoder is the self-attention mechanism. This mechanism first transforms the input vector into Q, K, and V, then calculates the input vector's self-attention weights. The hidden layer size in the feed- forward network inside the transformer is 32, activation function is $ReLU$. (3) CNN Layer: We employ the convolution kernels to extract local features of reentrancy vulnerability code slices. The CNN layer can extract the local features of code slices. After using convolution kernels to extract multiple features of code slices, we use pooling techniques to reduce dimensions, aiming to extract the most important features. In our study, we utilized 6 convolution kernels of size 2 and $AveragePooling$ to obtain the average features from the convolution kernels. (4) Classification Layer: After extracting code slice features with convolution kernels and pooling techniques, we feed them into a multi-layer perceptron (MLP; see classification layer in Fig. 6) for classification training, aiming to identify the presence of reentrancy vulnerabilities in the code. We utilize the $softmax$ as the activation function and cross-entropy as the loss function.

3.2.2 Detection result analysis

The deep learning network assigns weights to tokens in the input code slices during training. Therefore, we extract code lines with high attention scores from the deep learning network. The step (see mapping to code gadget in Fig. 4)illustrates an example of extracting attention scores from code lines. We first analyze the score distribution of all tokens in a code slice from the convolutional layer of the neural network. Then we map the scores to the code gadget. Finally, we then extract the high-attention vulnerable code lines.

3.3 Vulnerability repair phase

3.3.1 LLM-based repair

After obtaining the DL-based result analysis, we get the detection result and vulnerable code lines. We then propose a knowledge-aware, prompt-based approach to repair reentrancy vulnerability.

Knowledge-aware prompt-based vulnerability repair consists of four parts (see vulnerability repair prompt in Fig. 4): vulnerability examples, repaired vulnerability examples, repair approaches, and attention scores-based code lines. (1) Vulnerability example: The function-level code slice contains a reentrancy vulnerability. (2) Repaired example: The function-level code slice has been repaired using three various methods. (3) Repair approaches: These include locking the $call.value$ code line, replacing the $call.value$ function with transfer or send functions, and using the CEI pattern to ensure all parameters are properly checked before modifying the contract state (see vulnerability approach and case in Fig. 4). (4) DL-based result analysis: We obtain the attention score-based code lines from Sect. 3.2.2 (see detection result analysis in Fig. 4) and construct them into the repair prompt. This approach can help LLMs better locate the reentrancy vulnerability.

3.3.2 Repair result analysis

We follow the structured methodology to evaluate the effectiveness of large language models in repairing reentrancy vulnerabilities. Initially, we provide the LLMs with a code slice containing a reentrancy vulnerability accompanied by a knowledge-aware repair prompt. We then assess the LLMs' ability to produce the repaired version of the code. Following this, we rigorously verify the correctness of the repaired code. Additionally, we also integrate formal verification tools (e.g., Slither) to verify the functional equivalence of the repaired code. Finally, we summarize the LLMs' performance in addressing reentrancy vulnerabilities by calculating the repair success rate and the accuracy of the corrected outputs.

4 Experiment

This section first explains our experimental setting, and then we analyze the experimental results by answering the following three questions to assess the effectiveness of our approaches.

- RQ1: How does our approach perform compared with the existing state-of-the-art methods?
- RQ2: How do our proposed components perform in the ablation study?
- RQ3: What's the usefulness of our approach?

4.1 Experiment setting

Our experiment includes four crucial settings: datasets, baselines, experiment setup, and evaluation metrics.

Datasets: We utilize the ESC (Qian et al. 2020, 2023; Liu et al. 2021a), SmartBugs (Ferreira et al. 2021), and Reentrancy-Study (Zheng et al. 2023) datasets, which contain smart contract source codes from Ethereum.

Baselines: We select three types of tools for model comparison: rule-driven tools, deep learning-driven tools, and LLM-based tools to detect and repair reentrancy vulnerability.

- **Securify** (Tsankov et al. 2018) is a static analysis tool with formal verification designed to detect security vulnerabilities in Ethereum smart contracts.
- **Smartcheck** (Tikhomirov et al. 2018) is a tool for static analysis of Ethereum smart contracts that identifies security vulnerabilities.
- **Mythril** (Ferreira et al. 2021) is a security analysis tool for Ethereum smart contracts that leverages symbolic execution.
- **Oyente** (Luu et al. 2016) is a security analysis tool for detecting vulnerabilities in Ethereum smart contracts using symbolic execution.
- **Slither** (Feist et al. 2019) is a static analysis tool for Solidity code that identifies security vulnerabilities.
- **Osiris** (Torres et al. 2018) is a security analysis tool that combines static analysis and symbolic execution to identify vulnerabilities in smart contracts.
- **SmartFix** (So and Oh 2023) is an automatic vulnerability repair tool for Ethereum smart contracts.
- **sGuard** (Nguyen et al. 2021) is a run-time analysis tool to repair vulnerabilities in Ethereum smart contracts using a combination of static and dynamic analysis.
- **Elysium** (Ferreira Torres et al. 2022) is context-aware bytecode-level patching to automatically repair vulnerable smart contracts.
- **TMP** (Zhuang et al. 2020) is a graph neural network-based framework with a message propagation phase and a readout phase to detect smart contract vulnerability.
- **AME** (Liu et al. 2021a) is an approach for detecting vulnerabilities in smart contracts with global graph features and local expert features.
- **CGE** (Liu et al. 2021b) is a technique that uses graph-based models with expert knowledge to identify vulnerabilities in Ethereum smart contracts.
- **ReVulDL** (Zhang, et al. 2023) is a deep learning-based approach designed to detect reentrancy vulnerabilities in Ethereum smart contracts.
- **ChatGLM** (Du, et al. 2022) is a large language model designed for general-purpose dialogue generation by IntelliSpec-AI.
- **GPT-3.5-Turbo** (Sun, et al. 2023) is a highly efficient language model by OpenAI, optimized for faster and cost-effective performance, capable of generating

human-like text across various tasks.

- **CodeLlama** (Rozière, et al. 2023) is an LLM-based model developed by Meta, designed to achieve state-of-the-art results in a wide array of natural language processing tasks.
- The Meta **Llama-3.1** (Grattafiori, et al. 2024) is a 70-billion-parameter generative model that has been both trained and instruction-tuned.
- **Qwen2** (Chu, et al. 2024) is a cutting-edge model pre-trained on an extensive dataset of up to 18 trillion tokens, representing a significant advancement over its predecessor, Qwen2.
- **DeepSeek-R1** (Lastname, et al. 2025) is designed to perform a wide range of natural language processing and code understanding tasks. It demonstrates strong capabilities in text generation, code completion, and logical reasoning.
- **Starcoder2-15B** (Lozhkov, et al. 2024) is a 15-billion-parameter code-specialized language model developed by BigCode, aimed at supporting software development and code-related AI tasks.

Experiment Setup: We utilize an Alibaba Cloud server as an experimental environment equipped with an Intel(R) Xeon(R) Platinum 8269CY CPU and 15 GB of memory. We utilize TensorFlow to build the deep-learning neural network. We split the training, validation, and test sets in an 8:1:1 ratio. Additionally, we set the learning rate to 0.002, dropout to 0.5, and batch size to 64.

Evaluation Metrics: We employ four evaluation metrics, i.e., accuracy, precision, recall, and F1 score, widely used in smart contract vulnerability detection (Zhuang et al. 2020; Liu et al. 2021a; Qian et al. 2023). Additionally, we also employ two metrics, i.e., fixed rates and correction rates, to evaluate vulnerability repair performance.

- **Accuracy** measures the proportion of correctly predicted instances (both true positives and true negatives) out of the total number of instances in the dataset.
- **Precision** measures the proportion of correctly predicted positive instances among all instances predicted as positive, with high precision indicating fewer false positives.
- **Recall** measures the proportion of correctly predicted positive instances among all actual positives, with high recall indicating the model's ability to identify the most positive instances.
- **F1-score** is the harmonic mean of precision and recall, providing a balanced evaluation. A high F1 score indicates a model with a good trade-off between accuracy and recall.
- The **fixed rate (FR)** measures the proportion of vulnerabilities the LLMs can attempt to repair among the total number of slices.
- The **correct rate (CR)** reflects the proportion of successfully repaired vulnerabilities among the total number of slices.

4.2 Answer to RQ1 (overall comparison)

In this section, we present a comprehensive comparison and analysis of our approach to detecting and repairing reentrancy vulnerabilities, benchmarking it against estab-

lished methods. For the detection phase, we evaluate our approach alongside LLM-based, rule-based, and deep learning (DL)-based methodologies. However, in the repair phase, the comparison focuses on rule-based and LLM-based approaches, as there are currently no suitable DL-based tools available for smart contract repair.

4.2.1 Detection comparison

Table 1 illustrates that our approach has outperformed the rule-based, DL-based, and LLM-based approaches in the detection phase. (1) First, we used the LLM-based approaches for experiments. We found that the detection performance for reentrancy vulnerabilities was poor, with accuracy, recall, precision, and F1 score values of only 50.96%, 17.31%, 52.94%, and 26.09%, respectively. Additionally, the experiment results of CodeLlama, ChatGLM, DeepSeek, StarCoder, Qwen, and LLaMA are consistent with ChatGPT. The four LLMs, despite achieving perfect recall, suffer from low precision and high false positive rates, making them less suitable for real-world deployment. These results show that LLMs perform poorly in vulnerability detection when used directly without fine-tuning or prompt engineering. This indicates limitations in their scalability for domain-specific tasks like smart contract analysis unless enhanced with domain knowledge. (2) Next, we compared our approach with the rule-based tools, and the results showed that our method outperformed the best-performing tool (SliSE) in F1 score. Rule-based approaches can offer explainability due to their reliance on predefined rules and static analysis techniques to detect known vulnerability pat- terns. However, they suffer from limited generalization capability, as they cannot detect novel or complex vulnerability patterns that deviate from predefined rules. (3) Finally, we compared our approach with DL-based approaches in detection. The DL-based approach achieves high accuracy, recall, precision, and F1 scores, demonstrating superior generalization capability in handling large datasets. For instance, AME achieved an accuracy of 90.19%, and ReVulDL showed the best performance in recall, precision, and F1 score, with values of 93%, 92%, and 93%, respectively. However, our method surpassed these deep learning methods in all aspects, with improvements of 3.51%, 2.31%, 0.42%, and 0.85% in accuracy, recall, precision, and F1 score, respectively.

4.2.2 Repair comparison

Table 2 demonstrates that our approach surpasses both LLM-based and rule-based approaches in fixing reentrancy vulnerabilities. Initially, we employed various tools to remediate these vulnerabilities and subsequently verified the accuracy of the repaired smart contracts. Among the conventional tools, Smartfix was the most effective, correcting 46 out of 52 vulnerabilities, all of which were successfully repaired. Utilizing ChatGPT without prior repair examples resulted in 36 fixed vulnerabilities, with 30 being successfully repaired. The introduction of fixed example prompts enhanced the effectiveness of LLMs, increasing the number of fixed vulnerabilities to 44, with 42 successfully repaired. Further experiments involved comparisons with CodeLlama-7B, CodeLlama-13B, and ChatGLM3-6B, detailed in Table 2. Incorpo-

rating our knowledge-aware, prompt-based approach with LLMs markedly improved the repair outcomes, fixing 49 vulnerabilities, with 47 successfully repaired.

Answer to RQ1: In conclusion, our approach has outperformed the state-of-the- art rule-based, DL-based, and LLM-based baselines in reentrancy vulnerability detection and repair.

4.3 Answer to RQ2 (ablation study)

In this section, we discuss the impact of our proposed component on reentrancy vulnerability detection and repair, including feature extraction, data balance, a DL-based pre-trained model, and a knowledge-aware prompt.

- **Source code:** This approach directly inputs smart contract source code into the DL-based model for vulnerability detection without any data preprocessing.
- **Single function:** This approach inputs smart contract function-level code slices into the DL-based model for vulnerability detection. The function-level code slices include the $call.value$ related functions.
- **Our approach:** This approach inputs our proposed control-flow algorithm-based (see Sect. 3.1.2) code slices into a DL-based model for vulnerability detection.

Ablation Study 1: Feature Extraction Table 3 illustrates that various feature extractions significantly affect the reentrancy vulnerability detection. Our proposed control-flow-based feature extraction and data balancing approaches can improve detection performance. For feature input, the accuracy achieved by source code slices, single-function code slices, and multi-function code slices is 83.28%, 88.93%, and 94.46%, respectively. Additionally, recall, precision, and F1 scores also indicate that training with control-flow-based code slices is significantly better than source code input and single-function code slices.

- **Imbalanced:** We utilize the unprocessed imbalanced dataset to train a DL-based model for vulnerability detection.
- **SMOTE-Based Balanced:** We utilize the SMOTE algorithm to balance our datasets before training a DL-based model for vulnerability detection.
- **Our approach:** We propose a data balance algorithm (see Sect. 3.1.3) to balance our datasets before training a DL-based model for vulnerability detection.

Ablation Study 2: Data Balance Table 4 shows that the data balance approach also affects the reentrancy vulnerability detection. For data balancing, the imbalanced dataset achieves an F1 score of 57.14% and an accuracy of 96.97%, indicating significant overfitting during model training. Conversely, the results trained with SMOTE

Table 3 The ablation study of feature extraction

Method	Accuracy(%)	Recall(%)	Precision(%)	F1(%)
Source code	83.28	84.48	82.63	84.78
Single function	88.93	93.33	85.82	89.42
Our approach	94.46	95.74	92.17	94.78

Table 4 The ablation study of data balance

Methods	Accuracy(%)	Recall(%)	Precision(%)	F1(%)
Imbalanced	96.97	61.54	81.39	57.14
SMOTE-Based Balanced	94.24	94.62	93.61	94.11
Our approach-Based Balanced	94.46	95.74	92.17	94.78

and our algorithm exceed 90% in both accuracy and F1 scores, demonstrating effective deep learning training. Therefore, we conclude that our data balancing method not only enhances the detection performance of reentrancy vulnerabilities but also mitigates the risk of overfitting in model training.

- **Word2Vec:** A pre-trained model that learns vector representations of words by analyzing large text corpora
- **BERT:** A transformer-based model that is pre-trained on a large corpus of text with a masked language model objective.
- **RoBERTa:** An improved version of BERT that Facebook AI developed.
- **CodeBERT:** A pre-trained model designed specifically for programming languages, built on the BERT architecture.
- **CodeBERTa:** A variant of RoBERTa tailored for code-related tasks.

Ablation Study 3: Pre-Trained Model Table 5 illustrates that different cross-language pre-trained models also impact reentrancy vulnerability detection performance. Pre-trained large language models effectively improve vulnerability detection compared to deep learning models. For instance, the detection accuracy and F1 score achieved by Word2Vec are only 92.48% and 92.56%, while the cross- language pre-trained model CodeBERT achieves an accuracy of 94.46% and an F1 score of 94.78%. Comparing the experimental results of BERT and CodeBERT, we also find that incorporating code training in pre-trained models can improve reentrancy detection performance to some extent.

- **GPT-3.5-Turbo-0shot:** The direct prompt doesn't include any vulnerability knowledge or cases.
- **GPT-3.5-Turbo-2shot:** The prompt only includes a vulnerable case and a repaired case.
- **Our approach:** Our proposed knowledge-aware approach includes DL-based result analysis, reentrancy vulnerability repair approaches, and cases.

Table 5 The ablation study of the pre-trained model (PTM)

PTM	Accuracy(%)	Recall(%)	Precision(%)	F1 score(%)
Word2vec	92.48	93.33	91.80	92.56
BERT	93.76	94.54	91.60	94.06
RoBERTa	93.71	95.97	90.65	94.17
CodeBERT	94.46	95.74	92.17	94.78
CodeBERTa	94.28	95.24	91.50	94.56

Ablation Study 4: Knowledge-Aware Prompt As demonstrated in Table 6, enhancing the input knowledge-aware prompts considerably improves the performance of LLMs in repairing vulnerabilities across the ESC, Smart-Bugs, and Reentrancy-Study datasets. Initially, in the zero-shot setting, where ChatGPT solely evaluates and analyzes vulnerable smart contracts from the ESC dataset, the results are suboptimal: GPT-3.5-Turbo successfully repairs 30 code slices, only 23 of which are correct. How- ever, significant improvements are observed when ChatGPT is provided with examples of repairs and corresponding methods in the prompt (2-shot), leading to the successful repair of 44 code slices, of which 42 are accurately repaired. The most pronounced enhancement is noted when GPT-3.5-Turbo is further equipped with a knowledge- aware prompt, culminating in the successful repair of 48 code slices, with 47 being correct. In particular, the results on the Smart-Bugs and Reentrancy-Study datasets align with those observed in the ESC dataset. These findings underscore the efficacy of incorporating vulnerability examples into the model's inputs, as well as the utility of attention scores-based code slices in enhancing the model's ability to locate and amend vulnerabilities precisely.

Finally, Fig. 7 shows a vulnerable code fragment repaired by ChatGPT utilizing three approaches: locking-based, replacing-based, and checks-effects-interactions repair. The vulnerability in line 4 can cause a DAO attack (see source code in Fig. 7). When we seek help from ChatGPT, it can replace the $call.value$ function with the transfer function in (b) (see replacing in Fig. 7) or use lock-based repair in (c) (see locking in Fig. 7). Finally, in (d) (see CEI in Fig. 7), the model uses the checks- effects-interactions method to repair the vulnerability: first modifying the contract state in line 4, then interacting with the external contract in line 5, and finally verifying the success of the interaction in line 6. Thus, we conclude that designing specific prompts for adjustment is beneficial for repairing vulnerabilities, eliminating the need for complex algorithms to repair smart contract vulnerabilities.

Answer to RQ2: In conclusion, the ablation study validates the effectiveness of each component, including feature extraction, data balance, the pre-trained model-based DL network, and the knowledge-aware prompt.

Table 6 The ablation study of prompt type on ESC, SmartBugs, and ReentrancyStudy datasets

Dataset		fixed	FR(%)	Correct	CR(%)
ESC	GPT-3.5-Turbo-0shot	30/52	57.69	23/52	44.23
	GPT-3.5-Turbo-2shot	44/52	84.61	42/52	80.77
	Our approach	49/52	94.23(9.62↑)	47/52	90.38(↑)
SmartBugs	GPT-3.5-Turbo-0shot	72/174	41.38	63/174	36.21
	GPT-3.5-Turbo-2shot	119/174	68.39	106/174	60.02
	Our approach	156/174	89.66(21.27↑)	140/174	80.46(20.44↑)
ReentrancyStudy	GPT-3.5-Turbo-0shot	32/41	78.05	25/41	60.96
	GPT-3.5-Turbo-2shot	35/41	85.37	31/41	75.61
	Our approach	39/41	95.12(9.75↑)	37/41	90.24(14.63↑)

```
function withdraw(uint _amount) external {
    require(balances[msg.sender] >= _amount,
      "Insufficient balance");
    (bool success, ) = msg.sender.call.value(_amount);
    require(success, "Transfer failed");
    balances[msg.sender] -= _amount;
}
```

(a) Source Code

```
function withdraw(uint _amount) external {
    require(balances[msg.sender] >= _amount,
    "Insufficient balance");
    payable(msg.sender).transfer(_amount);
    balances[msg.sender] -= _amount;
}
```

(b) Replacing

```
bool private isTransferring;
function withdraw(uint _amount) external {
    require(balances[msg.sender] >= _amount,
          "Insufficient balance");
    isTransferring = true;
    (bool success, ) = msg.sender.call.value(_amount);
    require(success, "Transfer failed");
    balances[msg.sender] -= _amount;
    isTransferring = false;
}
```

(c) Locking

```
function withdraw(uint _amount) external {
    require(balances[msg.sender] >= _amount,
          "Insufficient balance");
    balances[msg.sender] -= _amount;
    (bool success,) = msg.sender.call{value: _amount}("");
    require(success, "Transfer failed");
}
```

(d) Check-Effect-Interactions (CEI)

Fig. 7 The examples of source code and three fixed examples

4.4 Answer to RQ3 (user study)

4.4.1 Design

We selected 12 smart contracts from the ESC dataset for the user study, including six vulnerable and six non-vulnerable contracts. As shown in Table 7, these smart contracts are categorized into three levels of difficulty: easy, medium, and difficult. Easy-level contracts featured simple code structures with straightforward function calls. Medium-level contracts had moderately complex logic, including conditional statements and multiple external calls. Difficult-level contracts exhibited highly complex code structures with nested function calls and intricate interactions between contract components. As shown in Table 7, the selected contracts included both vulnerable

Table 7 The smart contract scenarios in our user study

Task	Vul	Code Line	Difficulty
T-1	Yes	msg.sender.call.value(am)	Medium
T-2	Yes	msg.sender.call.value(am)	Medium
T-3	Yes	msg.sender.call.value(amount)()	Easy
T-4	Yes	msg.sender.call.value(weiToWithdraw)()	Easy
T-5	No	msg.sender.transfer(calculatedFibNumber *1 ether)	Medium
T-6	No	adr.call.value(msg.value)(data)	Medium
T-7	No	recipient.transfer(this.balance);	Easy
T-8	No	msg.sender.transfer(this.balance)	Easy
T-9	No	giveEthCharityAddress.call.value(ethToPay).gas(400,000)()	Difficult
T-10	No	! to.call.value (value)(data)	Difficult
T-11	Yes	etherHero.call.value(address(this).balance).gas(estGas)()	Difficult
T-12	Yes	p.recipient.call.value(p.amount)(transactionByteCode)	Difficult

and non-vulnerable lines of code involving ether transfers, ensuring representation across different vulnerability scenarios.

For the user study, we recruited six graduate students from our university, none of whom were involved in the research. These students are proficient in computer science and have basic knowledge of smart contracts, making them well-suited for the experiment. The students were asked to complete the study first with ChatGPT's assistance (G-1) and then with our knowledge-aware approach (G-2). Finally, we analyzed and compared the detection and repair reports to evaluate the effectiveness of our proposed approach.

4.4.2 Result

Table 8 presents the results of a user study comparing two experimental groups, G1 and G2, across 12 tasks. The evaluated metrics include detection accuracy, repair accuracy, task completion time (in seconds), and perceived usefulness, with improvements (Imp.) calculated as the difference in performance between G2 and G1. Upon analyzing the data, several important findings emerge. First, in terms of detection accuracy, G2 demonstrates an average improvement of 33.33% over G1, achieving higher detection accuracy across nearly all tasks. This suggests that G2 generally exhibits superior detection capabilities, likely due to an enhanced algorithm or methodology applied in G2. In contrast, repair accuracy shows a much more significant improvement, with G2 outperforming G1 by an impressive 56.7% on average. This indicates that G2 not only excels at identifying vulnerabilities but also offers more effective solutions for repairing them. When considering the time taken to complete the tasks, G2 consistently demonstrates faster completion times, with an average reduction of 14.9% compared to G1. This notable decrease in time is especially pronounced in tasks such as T-3 and T-4, where the difference is most significant, suggesting that G2's approach incorporates optimizations that streamline the overall process. Regarding usefulness, G2 again outperforms G1, with an average improvement of 50%. Although the increase in usefulness is relatively modest, it still suggests that users found G2 to be slightly more intuitive or effective in addressing their needs. This could be attributed to factors such as improved user interface design, more accurate results, or enhanced overall system performance.

Answer to RQ3: The combined improvements in detection and repair accuracy, faster task completion, and higher perceived usefulness indicate that our approach is a superior choice for detecting and repairing reentrancy vulnerability.

5 Threat to validity

Vulnerability Types: Our approach is exclusively applied to reentrancy vulnerability. As the Ethereum ecosystem continues to reveal more vulnerabilities, it is imperative to conduct further research on diverse vulnerabilities, including timestamp dependency, infinite loop, access control, and unchecked external calls vulnerability. Additionally, reentrancy vulnerabilities can stem from intricate sources, and existing works have limited capability (Zheng et al. 2023). Therefore, researchers should

Table 8 The result of the user study

Task	Detection G1	G2	Imp	Repair G1	G2	Imp	Time G1(s)	G2(s)	Imp	Usefulness G1	G2	Imp
T-1	100%	100%	0	60%	100%	40%	224	162	−27.7%	2.3	4.7	104.3%
T-2	100%	100%	0	40%	80%	40%	241.4	190	−21.3%	2.7	4.3	59.3%
T-3	100%	100%	0	60%	100%	40%	110.6	84	−24.1%	1.7	4	135.3%
T-4	100%	100%	0	60%	100%	40%	114	80	−29.8%	1.7	4.3	152.9%
T-5	0	60%	60%	0	60%	60%	204.8	170	−17%	2	2.7	35%
T-6	0	80%	80%	0	60%	60%	196.4	166	−15.5%	2	2.7	35%
T-7	40%	80%	40%	0	80%	80%	86	73.6	−14.4%	2.3	2.7	17.4%
T-8	40%	80%	40%	0	80%	80%	72	67	−6.9%	2.3	2.7	17.4%
T-9	0	100%	100%	0	80%	80%	381	369	−3.1%	2.3	3	30.4%
T-10	40%	80%	40%	0	80%	80%	311.2	219	−29.6%	2	2.3	15%
T-11	80%	100%	20%	40%	100%	60%	335.8	312.8	−6.8%	3.3	4	21.2%
T-12	80%	100%	20%	40%	60%	20%	411.2	394	−4.2%	1.7	2.7	58.8%
Average	56.7%	90%	33.3%	25%	81.7%	56.7%	224	190.6	−14.9%	2.2	3.3	50%

uncover and detect more reentrancy patterns beyond those solely associated with $call.value$.

LLM-driven Safety: We have recently implemented limited knowledge-aware prompt tuning to enhance the performance of LLMs. However, given their limited feature coverage of reentrancy vulnerabilities, there must be more untapped potential that can be found by utilizing the large language model. Additionally, various rules and algorithms are available to extract multi-function code slices and augment the imbalanced datasets, which restrict the application of LLMs. Therefore, incorporating LLMs into the solution may offer a more convenient approach to solving this problem.

6 Related work

6.1 Smart contract vulnerability

Smart contracts are immutable once deployed, a defining characteristic that sets them apart from programs written in other programming languages (Pan et al. 2023). Despite this immutability, security concerns surrounding smart contracts persist, largely due to frequent hacking incidents and inherent logical vulnerabilities. Recent research on smart contract security can be categorized into three distinct layers, based on the origin of vulnerabilities (Chen et al. 2020): (1) Solidity Layer: Vulnerabilities at this layer arise from logical errors in the smart contract programming language itself. Notable examples include reentrancy, integer overflow, access control issues, and improper exception handling. (2) EVM Layer: Vulnerabilities at the Ethereum Virtual Machine (EVM) layer are a result of the platform's unique requirements and characteristics. Key vulnerabilities in this category include the short address issue, misuse of $TX.origin$, and Ethereum loss due to gas manipulation. (3) Blockchain Layer: Vulnerabilities at the blockchain layer occur during inter-node communication within the blockchain network. Noteworthy vulnerabilities in this layer include timestamp dependencies, order dependencies, and transaction dependencies.

6.2 Detection and repair tool

Previous tools for detecting and mitigating reentrancy vulnerabilities can be broadly categorized into two types: rule-based (Feist et al. 2019; Kalra et al. 2018; Zhang et al. 2019; Luu et al. 2016) and deep learning- based approaches. Prominent tools in these categories include Oyente (Luu et al. 2016; Zhuang et al. 2020; Liu et al. 2021b; Li et al. 2023), Slither (Feist et al. 2019), Securify (Tsankov et al. 2018), SmartCheck (Tikhomirov et al. 2018), SliSE (Wang et al. 2024), Mythril (Ferreira et al. 2021), EVMPatch (Rodler et al. 2021), SMARTSHIELD (Zhang et al. 2020), Rechecker (Qian et al. 2020), TMP (Zhuang et al. 2020), AME (Liu et al. 2021a), CGE (Liu et al. 2021b), DeeSCVHunter (Yu et al. 2021), and DMT (Qian et al. 2023). For instance, (Zhuang et al. 2020) proposed a graph neural network (GNN)-based approach for detecting three common smart contract vulnerabilities, transforming basic functions, relevant variables, and callback functions into key node features. Liu et al. (2021a)

introduced a method that combines expert knowledge with graph neural networks for vulnerability detection, enabling the extraction of both global graph-level and local expert-level vulnerability features. In a similar vein, (Yu et al. 2020) proposed a gas-aware approach for repairing smart contract vulnerabilities, which adjusts code patches based on their gas consumption.

6.3 LLM-based software engineering

ChatGPT, developed by OpenAI (Radford et al. 2019), gained significant attention in 2022 for its advanced text generation capabilities (Brown et al. 2020). As large language models (LLMs) increasingly influence AI, researchers have begun applying these models to software engineering Chen et al. 2024; Hillebrand, et al. 2023; Min, et al. 2022). For instance, Chen utilized prompt-based GPT-3.5 and GPT-4 to detect vulnerabilities in the Smart-Bugs dataset. Although LLMs show high recall in identifying vulnerabilities, their precision remains limited. The primary causes of false positives include biases in protected mechanisms, misinterpretation of code semantics, distractions from comments, and interference from unexecuted code. Additionally, Ren (Ren et al. 2023) proposes a knowledge-driven chain with iterative check-rewrite steps and fine-grained knowledge-driven prompts to enhance code generation.

7 Conclusion

In this paper, we introduce a unified hybrid framework that integrates deep learning (DL)-based and knowledge-aware LLM-based methodologies to improve reentrancy vulnerability detection and repair. In the vulnerability detection phase, we construct a domain-specific deep-learning model with excellent feature extraction and data balance. This approach can achieve remarkable detection performance compared to direct prompt-based smart contract vulnerability detection. In the vulnerability repair phase, we extract the vulnerable code lines from the deep learning model and construct a knowledge-aware prompt for reentrancy vulnerability repair. Our comprehensive experimental evaluation demonstrates that our framework surpasses existing benchmarks, thereby significantly advancing the security framework for blockchain systems. The ablation study validates the effectiveness of each component, and the user study shows that our approach can guide the researcher to detect and repair vulnerabilities.

In future work, we will extend our model's capabilities to other LLM-driven smart contract vulnerabilities and explore advanced optimization techniques for enhancing the reasoning abilities of large language models in this domain.

Acknowledgements This research was supported by the Fundamental Research Funds for the Central Universities 226-2025-00004, and the National Natural Science Foundation of China under Grant No.62302437.

Author contribution Mengliang Li wrote the main manuscript text, Qiang Shen finish the experiment, Xiaoxue Ren, Han Fu, Zhuo Li, Jianling Sun review the manuscript.

Data availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare no competing interests.

References

Brown, T., et al.: Language Models are Few-Shot Learners. Adv. Neural. Inf. Process. Syst. **33**, 1877–1901 (2020)

Chen, C., et al.: When ChatGPT Meets Smart Contract Vulnerability Detection: How Far Are We? ACM Trans. Softw. Eng. Methodol. (2024). https://doi.org/10.1145/3702973

Chen, T. et al, "SODA: A Generic Online Detection Framework for Smart Contracts," in Proceedings 2020 Network and Distributed System Security Symposium, San Diego, CA: Internet Society, (2020) https://doi.org/10.14722/ndss.2020.24449

Chu, Y. et al., "Qwen2-Audio Technical Report," Jul. 15, 2024, https://doi.org/10.48550/arXiv.2407.10759.

David, I., Zhou, L., Qin, K., Song, D., Cavallaro, L., and Gervais, A."Do you still need a manual smart contract audit?." Jun. 22, 2023, https://doi.org/10.48550/arXiv.2306.12338.

DeepSeek-AI et al., "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," Jan. 22, 2025, https://doi.org/10.48550/arXiv.2501.12948.

Dong, Q. et al., "A Survey on In-context Learning," Dec. 31, 2022. Accessed: Dec. 13, 2023. [Online]. Available: http://arxiv.org/abs/2301.00234

Du, Z. et al., "GLM: General Language Model Pretraining with Autoregressive Blank Infilling," Mar. 17, 2022, https://doi.org/10.48550/arXiv.2103.10360.

Feist, J., Grieco, G., and Groce, A., "Slither: A Static Analysis Framework for Smart Contracts," in 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB), 8–15 (2019) https://doi.org/10.1109/WETSEB.2019.00008.

Ferreira, J.F., Cruz, P., Durieux, T., and Abreu, R., "SmartBugs: a framework to analyze solidity smart contracts," in Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering, in ASE '20. New York, NY, USA: Association for Computing Machinery, pp. 1349–1352. (2021) https://doi.org/10.1145/3324884.3415298.

Ferreira Torres, C., Jonker, H., State, R. "Elysium: Context-Aware Bytecode-Level Patching to Automatically Heal Vulnerable Smart Contracts," in Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses, in RAID '22. New York, NY, USA: Association for Computing Machinery, Oct. 2022, pp. 115–128. https://doi.org/10.1145/3545948.3545975.

M. Fu, C. Tantithamthavorn, V. Nguyen, and T. Le, "ChatGPT for Vulnerability Detection, Classification, and Repair: How Far Are We?," Oct. 15, 2023, https://doi.org/10.48550/arXiv.2310.09810.

Grattafiori A. et al., "The Llama 3 Herd of Models," Jul. 01, 2024, https://doi.org/10.48550/arXiv.2407.21783.

Hillebrand, L., et al., "Improving Zero-Shot Text Matching for Financial Auditing with Large Language Models," Aug. 14, 2023, https://doi.org/10.48550/arXiv.2308.06111.

Hu, S., Huang, T., İlhan, F. Tekin, S.F. and Liu, L."Large Language Model-Powered Smart Contract Vulnerability Detection: New Perspectives," Oct. 16, 2023, https://doi.org/10.48550/arXiv.2310.01152.

Kalra, S., Goel, S., Dhawan, M., and Sharma, S. "ZEUS: Analyzing Safety of Smart Contracts," in Proceedings 2018 Network and Distributed System Security Symposium, San Diego, CA: Internet Society (2018) https://doi.org/10.14722/ndss.2018.23082.

Li, M., Ren, X., Fu, H., Li, Z., Sun, J. "ConvMHSA-SCVD: Enhancing Smart Contract Vulnerability Detection through a Knowledge-Driven and Data-Driven Framework," in 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE), Oct. 2023, pp. 578–589. https://doi.org/10.1109/ISSRE59848.2023.00025.

Liu, Z., Qian, P., Wang, X., Zhuang, Y., Qiu, L., Wang, X."Combining Graph Neural Networks with Expert Knowledge for Smart Contract Vulnerability Detection," IEEE Transactions on Knowledge and Data Engineering, pp. 1–1 (2021) https://doi.org/10.1109/TKDE.2021.3095196.

Liu, Z., Qian, P., Wang, X., Zhu, L., He, Q., Ji, S. "Smart Contract Vulnerability Detection: From Pure Neural Network to Interpretable Graph Feature and Expert Pattern Fusion," [cs], Jun. 2021, Accessed: Oct. 27, 2021. [Online]. Available: http://arxiv.org/abs/2106.09282

Luu, L., Chu, D.-H., Olickel, H., Saxena, P., and Hobor, A. "Making Smart Contracts Smarter," in Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, in CCS '16. New York, NY, USA: Association for Computing Machinery, pp. 254–269. (2016) https://doi.org/10.1145/2976749.2978309.

Lozhkov, A. et al., "StarCoder 2 and The Stack v2: The Next Generation," Feb. 29, 2024, https://doi.org/10.48550/arXiv.2402.19173

Min, S., et al., "Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?," in Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, Y. Goldberg, Z. Kozareva, and Y. Zhang, Eds., Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, Dec. 2022, pp. 11048–11064. https://doi.org/10.18653/v1/2022.emnlp-main.759.

Nguyen, T.D., Pham, L.H., Sun, J. "SGUARD: Towards Fixing Vulnerable Smart Contracts Automatically," in 2021 IEEE Symposium on Security and Privacy (SP), pp. 1215–1229. (2021) https://doi.org/10.1109/SP40001.2021.00057.

Pan, Y., Xu, Z., Li. L.T., Yang, Y., and Zhang, M. "Automated Generation of Security-Centric Descriptions for Smart Contract Bytecode," in Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, Seattle WA USA: ACM, pp. 1244–1256. (2023) https://doi.org/10.1145/3597926.3598132.

Qian, P., Liu, Z., Yin, Y., He, Q. "Cross-Modality Mutual Learning for Enhancing Smart Contract Vulnerability Detection on Bytecode," in Proceedings of the ACM Web Conference 2023, in WWW '23. New York, NY, USA: Association for Computing Machinery, pp. 2220–2229. (2023) https://doi.org/10.1145/3543507.3583367.

Qian, P., Liu, Z., He, Q., Zimmermann, R., Wang, X.: Towards automated reentrancy detection for smart contracts based on sequential models. IEEE Access **8**, 19685–19695 (2020). https://doi.org/10.1109/ACCESS.2020.2969429

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I. "Language Models are Unsupervised Multitask Learners".

Ren, X., Ye, X., Zhao, D., Xing, Z., Yang, X. "From Misuse to Mastery: Enhancing Code Generation with Knowledge-Driven AI Chaining," Sep. 27, 2023, https://doi.org/10.48550/arXiv.2309.15606.

Rodler, M., Li, W., Karame, G.O., Davi, L. "EVMPatch: Timely and Automated Patching of Ethereum Smart Contracts," presented at the 30th {USENIX} Security Symposium ({USENIX} Security 21), pp. 1289–1306. (2021) Accessed: Oct. 08, 2021. [Online]. https://www.usenix.org/conference/usenixsecurity21/presentation/rodler

Rozière, B. et al., "Code Llama: Open Foundation Models for Code," Aug. 25, 2023, https://doi.org/10.48550/arXiv.2308.12950.

Shakya, S., Mukherjee, A., Halder, R., Maiti, A., Chaturvedi, A. "SmartMixModel: Machine Learning-based Vulnerability Detection of Solidity Smart Contracts," in 2022 IEEE International Conference on Blockchain (Blockchain), pp. 37–44. (2022) https://doi.org/10.1109/Blockchain55522.2022.00016.

So, S. and Oh, H.,"SmartFix: Fixing Vulnerable Smart Contracts by Accelerating Generate-and-Verify Repair using Statistical Models," in Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, San Francisco CA USA: ACM, pp. 185–197. (2023) https://doi.org/10.1145/3611643.3616341.

Sun, Y. et al., "When GPT Meets Program Analysis: Towards Intelligent Detection of Smart Contract Logic Vulnerabilities in GPTScan," Aug. 07, 2023, https://doi.org/10.48550/arXiv.2308.03314.

Tikhomirov, S., Voskresenskaya, E., Ivanitskiy, I., Takhaviev, R., Marchenko, E. and Alexandrov, "SmartCheck: static analysis of ethereum smart contracts," in Proceedings of the 1st International Workshop on Emerging Trends in Software Engineering for Blockchain, in WETSEB '18. New York, NY, USA: Association for Computing Machinery, pp. 9–16. (2018) https://doi.org/10.1145/3194113.3194115.

Torres, C.F., Schütte, J., State, R. "Osiris: Hunting for Integer Bugs in Ethereum Smart Contracts," in Proceedings of the 34th Annual Computer Security Applications Conference, in ACSAC '18. New York, NY, USA: Association for Computing Machinery, pp. 664–676. (2018) https://doi.org/10.1145/3274694.3274737.

Tsankov, P., Dan, A., Drachsler-Cohen, D., Gervais, A., Bünzli, F., Vechev, M. "Securify: Practical Security Analysis of Smart Contracts," in Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, in CCS '18. New York, NY, USA: Association for Computing Machinery, pp. 67–82. (2018) https://doi.org/10.1145/3243734.3243780.

Wang, Z., Chen, J., Wang, Y., Zhang, Y., Zhang, W., Zheng, Z."Efficiently Detecting Reentrancy Vulnerabilities in Complex Smart Contracts," Proc. ACM Softw. Eng., vol. 1, no. FSE, p. 8:161–8:181, (2024) https://doi.org/10.1145/3643734.

Xue, Y., Ma, M., Lin, Y., Sui, Y., Ye, J., and Peng, T., "Cross-Contract Static Analysis for Detecting Practical Reentrancy Vulnerabilities in Smart Contracts," in 2020 35th IEEE/ACM International Conference on Automated Software Engineering (ASE) pp. 1029–1040 (2020) Accessed: Jun. 07, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/9286095

X. Yu, H. Zhao, B. Hou, Z. Ying, and B. Wu, "DeeSCVHunter: A Deep Learning-Based Framework for Smart Contract Vulnerability Detection," in 2021 International Joint Conference on Neural Networks (IJCNN), pp. 1–8. (2021) https://doi.org/10.1109/IJCNN52387.2021.9534324.

Yu, X.L., Al-Bataineh, O., Lo, D., Roychoudhury, A., "Smart Contract Repair," May 20, 2020. Accessed: Jun. 14, 2023. [Online]. Available: http://arxiv.org/abs/1912.05823

Zhang, P., Xiao, F., and Luo, X., "SolidityCheck : Quickly Detecting Smart Contract Problems Through Regular Expressions," (2019) https://doi.org/10.48550/arXiv.1911.09425.

Zhang, Y., Ma, S., Li, J., Li, K. Nepal, S. Gu, D. "SMARTSHIELD: Automatic Smart Contract Protection Made Easy," in 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER), Feb. 2020, pp. 23–34. https://doi.org/10.1109/SANER48275.2020.9054825.

Zhang, Z., et al., "Reentrancy Vulnerability Detection and Localization: A Deep Learning Based Two-phase Approach," in Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, in ASE '22. New York, NY, USA: Association for Computing Machinery, pp. 1–13. 2023 https://doi.org/10.1145/3551349.3560428.

Zheng, Z., Zhang, N., Su, J., Zhong, Z., Ye, M., Chen, J., "Turn the Rudder: A Beacon of Reentrancy Detection for Smart Contracts on Ethereum," in 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE) pp. 295–306. (2023) https://doi.org/10.1109/ICSE48619.2023.00036.

Zhuang, Y., Liu, Z., Qian, P., Liu, Q., Wang, X., He, Q. "Smart Contract Vulnerability Detection using Graph Neural Network," in Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, Yokohama, Japan: International Joint Conferences on Artificial Intelligence Organization, pp. 3283–3290 (2020) https://doi.org/10.24963/ijcai.2020/454.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Mengliang Li[1] · Qiang Shen[2] · Xiaoxue Ren[1] · Han Fu[3] · Zhuo Li[4] · Jianling Sun[3]

✉ Xiaoxue Ren
xxren@zju.edu.cn

Mengliang Li
123lmliang@zju.edu.cn

Qiang Shen
qiangq688@gamil.com

Han Fu
11821003@zju.edu.cn

Zhuo Li
lizhuo@zju.edu.cn

Jianling Sun
sunjl@zju.edu.cn

1 Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, Zhejiang University, Hangzhou, Zhejiang, China

2 College of Software Technology, Zhejiang University, Hangzhou, Zhejiang, China

3 College of Computer Science and Technology, Zhejiang University, Hangzhou, Zhejiang, China

4 State Street Technology Ltd, Hang Zhou, China